Практика 1. Развёртывание учебной SIEM

Собираем учебную SIEM из двух программ, каждая в своём контейнере Docker:

Программа	Что делает	Адрес
Elasticsearch	хранит события и ищет по ним; управляется запросами по адресу	`localhost:9200`
Kibana	веб-интерфейс к Elasticsearch: поиск, графики, правила	`http://localhost:5601`

Что из чего состоит — описано в файле `docker-compose.yml` в репозитории. Нужно: Linux (или macOS, Windows + WSL 2), Docker с Compose, `git`, `curl`, `openssl`, 6 ГБ свободной памяти, 8 ГБ на диске.

Как читать команды

Терминал	Команду вводят (или вставляют) в терминал и нажимают Enter. Все команды — в папке репозитория
`\` в конце строки	Команда продолжается на следующей строке. Копировать всю команду целиком, до строки без `\`
`# текст`	Комментарий для человека. Терминал его не выполняет
`$ELASTIC_PASSWORD`	Подставить значение переменной — здесь ваш пароль из файла `.env`
`curl`	Программа, которая отправляет запрос по адресу и печатает ответ. Так управляют Elasticsearch: запрос по адресу `localhost:9200/…`

Шаг 1. Скачать практику и задать пароли

Зачем. Скачать файлы практики и придумать пароли, с которыми запустятся Elasticsearch и Kibana.

```
git clone https://github.com/ObeWan99/P1-normalization.git
cd P1-normalization
cp .env.example .env
openssl rand -hex 16
nano .env
source .env
```

Часть команды	Что значит
`git clone <адрес>`	Скачать репозиторий с GitHub. Появится папка `P1-normalization`
`cd P1-normalization`	Перейти в эту папку. Все следующие команды — отсюда
`cp .env.example .env`	Скопировать образец файла паролей в файл `.env`
`openssl rand -hex 16`	Напечатать случайную строку из 32 символов. Скопируйте её — это будет ключ шифрования
`nano .env`	Открыть `.env` в текстовом редакторе. Вписать значения после `=` (латиница и цифры, без пробелов) → **Ctrl+O**, Enter — сохранить → **Ctrl+X** — выйти
`source .env`	Прочитать `.env`: пароли станут переменными этого терминала. **В новом окне терминала — повторить**

Что вписать в `.env`:

`ELASTIC_PASSWORD`	пароль пользователя `elastic` — с ним вы входите в Kibana
`KIBANA_PASSWORD`	пароль служебной учётки `kibana_system`, под которой Kibana подключается к Elasticsearch
`ENCRYPTION_KEY`	строка из `openssl rand -hex 16` — Kibana шифрует ею свои настройки

Шаг 2. Запустить Elasticsearch

Зачем. Запустить хранилище событий и убедиться, что оно отвечает и пускает по паролю.

```
docker compose up -d elasticsearch
docker compose ps
curl -u "elastic:$ELASTIC_PASSWORD" localhost:9200
```

Часть команды	Что значит
`docker compose`	Программа, которая читает `docker-compose.yml` и управляет описанными там контейнерами
`up`	Запустить (при первом запуске сначала скачает образ, ≈1,6 ГБ)
`-d`	В фоне: контейнер работает, терминал свободен
`elasticsearch`	Только этот контейнер. Kibana запустим позже
`docker compose ps`	Показать контейнеры и их состояние. Повторять, пока не появится `(healthy)` — 1–2 минуты
`curl -u "elastic:$ELASTIC_PASSWORD"`	Отправить запрос от имени `elastic` с вашим паролем (`-u` — логин:пароль)
`localhost:9200`	Адрес Elasticsearch на этой машине (порт 9200)

✔ **Должно получиться:** `docker compose ps` — в столбце STATUS `(healthy)`; `curl` — ответ с `"number" : "9.4.8"`.
Пока в STATUS `(health: starting)`, `curl` отвечает ошибкой `Connection reset by peer` — Elasticsearch ещё запускается. Подождать и повторить.

Шаг 3. Задать пароль служебной учётке Kibana

Зачем. Kibana подключается к Elasticsearch не под `elastic`, а под служебной учёткой `kibana_system` с минимальными правами. У этой учётки пока нет пароля — задаём ей пароль из `KIBANA_PASSWORD`.

```
curl -u "elastic:$ELASTIC_PASSWORD" -H 'Content-Type: application/json' \
  -X POST localhost:9200/_security/user/kibana_system/_password \
  -d "{\"password\":\"$KIBANA_PASSWORD\"}"
```

Часть команды	Что значит
`curl -u "elastic:…"`	Запрос от имени `elastic`
`-H 'Content-Type: application/json'`	Заголовок запроса: «данные, которые я посылаю, — в формате JSON»
`-X POST`	Тип запроса POST — «выполни действие с моими данными»
`localhost:9200/_security/user/kibana_system/_password`	Адрес говорит, что делаем: безопасность → пользователь `kibana_system` → пароль
`-d "…"`	Сами данные: `{"password":"ваш пароль"}`. `\"` — кавычка внутри кавычек

✔ **Должно получиться:** `{}` — пустой ответ, значит без ошибок.

Шаг 4. Запустить Kibana

Зачем. Запустить веб-интерфейс.

```
docker compose up -d
```

Часть команды	Что значит
`docker compose up -d`	Запустить всё, что описано в `docker-compose.yml`. Elasticsearch уже работает — его не тронет, добавится Kibana (≈1,6 ГБ при первом запуске)

✔ **Должно получиться:** через 1–2 минуты в браузере открывается `http://localhost:5601`; логин `elastic`, пароль — ваш `ELASTIC_PASSWORD`. «Kibana server is not ready yet» — подождать минуту и обновить страницу.

Шаг 5. Загрузить шаблоны индексов

Зачем. Elasticsearch хранит события в **индексах** — это как таблицы в базе данных. Шаблон заранее говорит, какого типа будут поля: дата, число, IP-адрес. От типа зависит поиск: например, по полю типа IP можно искать целую подсеть. Шаблон `raw` — для сырых журналов, `norm` — для нормализованных.

```
curl -u "elastic:$ELASTIC_PASSWORD" -H 'Content-Type: application/json' \
  -X PUT localhost:9200/_index_template/raw -d @setup/raw-template.json
curl -u "elastic:$ELASTIC_PASSWORD" -H 'Content-Type: application/json' \
  -X PUT localhost:9200/_index_template/norm -d @setup/norm-template.json
```

Часть команды	Что значит
`-X PUT`	Тип запроса PUT — «создай или замени объект по этому адресу»
`localhost:9200/_index_template/raw`	Объект — шаблон индексов с именем `raw`
`-d @setup/raw-template.json`	Данные взять из файла (`@` — «из файла»). Файл лежит в репозитории, его можно открыть и посмотреть

Остальные части — как в шаге 3. Вторая команда — то же для шаблона `norm`.
✔ **Должно получиться:** `{"acknowledged":true}` — дважды.

Шаг 6. Загрузить журналы

Зачем. Положить три журнала в три индекса: `raw-auth` — входы, `raw-web` — веб-портал, `raw-system` — серверы. Загружаем через **Bulk API** — массовую загрузку: тысячи событий одним запросом. В файлах строки идут парами: служебная `{"index":{}}` («добавь в индекс») и само событие.

```
for j in auth web system; do
  curl -s -u "elastic:$ELASTIC_PASSWORD" -H 'Content-Type: application/x-ndjson' \
    -X POST "localhost:9200/raw-$j/_bulk?refresh=true" --data-binary @data/raw-$j.ndjson \
    | grep -o '"errors":[a-z]*'
done
```

Часть команды	Что значит
`for j in auth web system; do … done`	Повторить команду три раза. Вместо `$j` подставится по очереди `auth`, `web`, `system`
`-s`	Не показывать полосу загрузки
`-H 'Content-Type: application/x-ndjson'`	Данные — NDJSON: по одному JSON в строке
`-X POST "localhost:9200/raw-$j/_bulk"`	Массовая загрузка (`_bulk`) в индекс `raw-auth`, `raw-web`, `raw-system`. Индекс создастся сам — по шаблону `raw`
`?refresh=true`	Сразу сделать события доступными для поиска
`--data-binary @data/raw-$j.ndjson`	Данные — из файла журнала, байт в байт
`| grep -o '"errors":[a-z]*'`	Ответ огромный — по строке на событие. `|` передаёт его программе `grep`, та оставляет только `"errors":false` или `"errors":true`

Проверить, сколько событий легло:

```
curl -u "elastic:$ELASTIC_PASSWORD" \
  "localhost:9200/_cat/indices/raw-*?v&h=index,docs.count&s=index"
```

Часть команды	Что значит
`_cat/indices/raw-*`	Список индексов таблицей; только те, чьё имя начинается на `raw-`
`?v&h=index,docs.count&s=index`	`v` — с заголовками, `h=` — столбцы «имя» и «число событий», `s=` — сортировать по имени

✔ **Должно получиться:** `"errors":false` — три раза; в таблице `raw-auth` 20000, `raw-system` 15000, `raw-web` 15000.

Шаг 7. Нормализовать веб-журнал (готовый образец)

Зачем. Конвейер (ingest pipeline) — цепочка шагов, через которую проходит каждое событие: переименовать поля по стандарту ECS, привести время. Конвейер для веб-журнала готов — загружаем его и пропускаем через него журнал. Это образец: конвейеры для двух других журналов вы соберёте сами.

```
curl -u "elastic:$ELASTIC_PASSWORD" -H 'Content-Type: application/json' \
  -X PUT localhost:9200/_ingest/pipeline/norm-web -d @pipelines/norm-web.json
curl -u "elastic:$ELASTIC_PASSWORD" -H 'Content-Type: application/json' \
  -X POST "localhost:9200/_reindex?refresh=true" \
  -d '{"source":{"index":"raw-web"},"dest":{"index":"norm-web","pipeline":"norm-web"}}'
```

Часть команды	Что значит
`-X PUT …/_ingest/pipeline/norm-web`	Создать конвейер с именем `norm-web`
`-d @pipelines/norm-web.json`	Шаги конвейера — из файла (его можно открыть и посмотреть)
`-X POST …/_reindex`	**Reindex** — переложить события из одного индекса в другой
`"source":{"index":"raw-web"}`	откуда: сырой веб-журнал
`"dest":{"index":"norm-web", "pipeline":"norm-web"}`	куда: новый индекс `norm-web`, по дороге — через конвейер `norm-web`

✔ **Должно получиться:** первая команда — `{"acknowledged":true}`; вторая — `"created":15000` и `"failures":[]`.

Шаг 8. Настроить Kibana

Дальше — в браузере. Любой раздел Kibana находится через строку поиска вверху страницы.

Advanced Settings — изменить две настройки, внизу **Save changes**:

- **Timezone for date formatting** → `UTC`.
  Зачем: в журналах время в UTC; иначе Kibana покажет его в поясе браузера, и время не сойдётся с журналами.
- **Time filter defaults** → стереть содержимое и вставить:

```
{ "from": "2025-03-10T00:00:00Z", "to": "2025-03-17T00:00:00Z" }
```

  Зачем: журналы — за 10–16 марта 2025; иначе Kibana показывает последние 15 минут, и экран пустой.

Data Views → **Create data view** → **Save data view to Kibana**, пять раз. Представление данных говорит Kibana, из каких индексов брать события:

Name	Index pattern	Timestamp field
`Сырые журналы (raw-*)`	`raw-*`	`--- I don't want to use the time filter ---`
`Журнал входов (raw-auth)`	`raw-auth`	то же
`Журнал веб-портала (raw-web)`	`raw-web`	то же
`Журнал серверов (raw-system)`	`raw-system`	то же
`Нормализованные журналы (norm-*)`	`norm-*`	`@timestamp`

Name — название в списке, Index pattern — какие индексы (* — «любые символы»), Timestamp field — поле времени для фильтра по датам.

Проверка

✔ **Должно получиться:** Discover → слева вверху «Сырые журналы (raw-*)» — **50,000** событий; «Нормализованные журналы (norm-*)» — **15,000** (пока только веб).

Если что-то не так

Что видите	Что делать
`permission denied ... docker.sock`	нет прав на Docker: `sudo usermod -aG docker $USER`, выйти из системы и войти снова
`port is already allocated`	порт 9200 или 5601 занят другим контейнером: `docker ps` — посмотреть, `docker stop <имя>` — остановить
`curl: (56) Recv failure: Connection reset by peer`	Elasticsearch ещё запускается: `docker compose ps` — дождаться `(healthy)`, повторить
`401` / `unable to authenticate`	в этом терминале не выполнен `source .env`; или пароль в `.env` поменяли после первого запуска — начать заново
вместо ответа HTML-страница	мешает прокси: `unset http_proxy https_proxy`

Что видите	Что делать
Elasticsearch не `healthy`	посмотреть журнал контейнера: `docker compose logs elasticsearch --tail 30`; свободную память: `free -h`
Kibana «not ready yet» больше 5 минут	повторить шаг 3, затем `docker compose restart kibana`
`"errors":true` на шаге 6	не выполнен шаг 5 или журналы грузят второй раз — начать заново
Discover: 0 событий	период справа вверху: 10–16 марта 2025

Остановить и начать заново

`docker compose stop`	остановить контейнеры; данные сохранятся
`docker compose start`	запустить снова
`docker compose down -v`	**начать заново**: удалить контейнеры, а с `-v` — и данные. Затем с шага 2 (`.env` остаётся)